

CRITERION C: DEVELOPMENT

Introduction

This product is an Android to-do app called “Tick Off Time” designed in Android Studio and Kotlin. It allows creating, editing, and deleting categories and tasks that enable to planning of any activities. It includes a calendar, a list of tasks assigned for the present day, a list of tasks assigned for the present week, and a list of categories with all assigned tasks. Additionally, it has the option to find a task having provided the expected features.

List of used techniques:

- Object Oriented Programming structures
- Database and database operations
- Composable functions and components
- Dependency management
- Calendar and date operations
- Validation rules and conditional statements
- Navigation component
- Lists and arrays

Object Oriented Programming structures

In this programme, 4 main OOP structures were used:

1. Class
2. Data class
3. Object
4. Interface
5. Enum class

Class

Classes were used particularly due to their inheritance and polymorphism abilities. In the example below the class TaskEditViewModel as ViewModel() allows access to different functions responsible for different data operations.¹

¹. *Implement sell item*. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-update-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-update-data-room#7> . Read and update data with Room.

Figure 1. The graphic presenting the TaskEditViewModel() class

```
class TaskEditViewModel(
    savedStateHandle: SavedStateHandle,
    private val tasksRepository: TasksRepository
) : ViewModel() {

    var taskUiState by mutableStateOf(TaskUiState())
    private set

    private val taskId: Int = checkNotNull(savedStateHandle[TaskEditDestination.taskIdArg])

    init {
        viewModelScope.launch { this: CoroutineScope
            taskUiState = tasksRepository.getTasksStream(taskId) Flow<Task?>
                .filterNotNull() Flow<Task>
                .first() Task
                .toTaskUiState(actionEnabled = true)
        }
    }

    fun updateTaskUiState(newTaskUiState: TaskUiState) {
        taskUiState = newTaskUiState.copy( actionEnabled = newTaskUiState.isValid())
    }

    suspend fun updateTask() {
        if (taskUiState.isValid()) {
            tasksRepository.updateTask(taskUiState.toTask())
        }
    }

    suspend fun updateSpecificTask(task: Task) {
        tasksRepository.updateTask(task)
    }

    suspend fun deleteTask() {
        tasksRepository.deleteTask(taskUiState.toTask())
    }

    suspend fun deleteSpecificTask(task: Task) {
        tasksRepository.deleteTask(task)
    }
}
```

At a later stage, this class is used to edit tasks, for instance, in the EditingTaskMenu() function.²

Figure 2. The graphic presenting the EditingTaskMenu() function's parameters

```
@SuppressLint("UnusedMaterialScaffoldPaddingParameter")
@Composable
fun EditingTaskMenu(
    navController: NavController,
    viewModel: TaskEditViewModel = viewModel(factory = AppViewModelProvider.Factory),
    categoryViewModel: CategoryHomeViewModel = viewModel(factory = AppViewModelProvider.Factory)
){}
```

² Implement sell item. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-update-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-update-data-room#7> . Read and update data with Room.

Data class

Data classes were used to facilitate storing a few various values as one element. The use of it can be seen in declaring TaskUiState, which contains 9 values of different types.³

Figure 3. The graphic presenting the TaskUiState() data class

```
data class TaskUiState(  
    val id: Int = 0,  
    val name: String = "",  
    val description: String = "",  
    val category: String = "Uncategorised",  
    val type: String = "",  
    val firstDate: String = LocalDate.now().toString(),  
    val secondDate: String = LocalDate.now().plusDays( daysToAdd: 9999999).toString(),  
    val daysRepeated: Int = 1234567,  
    val actionEnabled: Boolean = false  
)
```

TaskUiState was used to store information from the user interface and convert it to Task, which is also a data class, and be saved properly.

Figure 4. The graphic presenting the Task() data class⁴

```
@Entity(tableName = "tasks")  
data class Task(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int,  
    val name: String,  
    val description: String,  
    val category: String, //change it later to classes  
    val type: String, //change it later to classes  
    val firstDate: String,  
    val secondDate: String,  
    val daysRepeated: Int,  
)
```

³ Add the save functionality. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#8> . Persist data with Room

⁴ Create an item entity. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#4> . Persist data with Room

Task and Category are very important data classes since databases are created based on them. They have significant features that allow to creation of elements with proper parameters.

Figure 5. The graphic presenting the Category() data class⁵

```
@Entity(tableName = "categories") //  
data class Category(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int,  
    val name: String,  
    val colour: Int,  
)
```

Object

Objects were used to facilitate navigating from the task or the category to its details. In the example below, the TaskEditDestination object, which contains 4 values, is presented. It allows you to proceed to a task's edit menu easily.⁶

Figure 6. The graphic presenting the TaskEditDestination object

```
object TaskEditDestination : NavigationDestination {  
    override val route = "task_edit"  
    override val titleRes = "Editing task"  
    const val taskIdArg = "categoryId"  
    val routeWithArgs = "$route/{$taskIdArg}"  
}
```

Interface

Interfaces were used to store different functions in one place. The use of it is presented below.

⁵ Create an item entity. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab=https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#4>. Persist data with Room

⁶ Dev Android. 2022. Basic Android Kotlin Compose Training Inventory App.

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git> (2023)

Figure 7. The graphic presenting the TaskDao() interface⁷

```
@Dao
interface TaskDao {
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun create(task: Task)

    @Update
    suspend fun update(task: Task)

    @Delete
    suspend fun delete(task: Task)

    @Query("SELECT * from tasks WHERE id= :id")
    fun getTask(id: Int): Flow<Task>

    @Query("SELECT * from tasks WHERE category= :category")
    fun getTaskByCategory(category: String): Flow<Task>

    @Query("SELECT * from tasks ORDER BY name ASC")
    fun getAllTasks(): Flow<List<Task>>
}
```

In this case, there are 6 functions responsible for database operations. They were used to create a connection with the database easily.

Enum class

An enum class was used only to store navigation destinations. Since it stores them, it avoids complications with incorrect values passing as a destination.

⁷ Create the item DAO. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#5>. Persist data with Room

Figure 8. The graphic presenting the Mains() enum class⁸

```
enum class Mains(@StringRes val title: Int) {  
    ListOfTasks(title = "List of tasks"),  
    Calendar(title = "Calendar"),  
    ToDoToday(title = "To do today"),  
    ToDoThisWeek(title = "To do this week"),  
    CreatingCategory(title = "Creating category"),  
    CreatingTask(title = "Creating task"),  
    Searching(title = "Search")  
}
```

Database and database operations

In this programme, there are two main databases - for tasks and for categories. They are initialised based on already created data classes (Figure 4., Figure 5.)

Figure 9. The graphic presenting the TaskDatabase() abstract class as a database⁹

```
@Database(entities = [Task::class], version = 1, exportSchema = false)  
abstract class TaskDatabase: RoomDatabase() {  
    abstract fun taskDao(): TaskDao  
    companion object {  
        @Volatile  
        private var Instance: TaskDatabase? = null  
  
        fun getDatabase(context: Context): TaskDatabase {  
            return Instance ?: synchronized( lock: this) {  
                Room.databaseBuilder(context, TaskDatabase::class.java, name: "task_database")  
                    .build() TaskDatabase  
                    .also {Instance = it}  
            }  
        }  
    }  
}
```

⁸ Define routes and create a NavHostController. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-navigation?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-4-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-navigation#3>. Navigate between screens with Compose

⁹ Create a Database instance. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#6>. Persist data with Room

Figure 10. The graphic presenting the `CategoryDatabase()` abstract class as a database¹⁰

```
@Database(entities = [Category::class], version = 1, exportSchema = false)
abstract class CategoryDatabase: RoomDatabase() {
    abstract fun categoryDao(): CategoryDao
    companion object {
        @Volatile
        private var Instance: CategoryDatabase? = null

        fun getDatabase(context: Context): CategoryDatabase {
            return Instance ?: synchronized(lock: this) {
                Room.databaseBuilder(context, CategoryDatabase::class.java, name: "category_database")
                    .build() CategoryDatabase
                    .also { Instance = it }
            }
        }
    }
}
```

Each data class has its values and therefore each object in the database has the same features. Since the whole app stores many tasks and categories that must be stored even if the app is closed, creating a database was the obligatory option.

Figure 11. The graphic presenting the database with the categories

categories

tasks

↺

Live updates

↗

Results are read-only

50

▼

▶▶

	🔗 id	name	colour
1	1	Category 1	-65536
2	2	Category 2	-16711936
3	3	Category 3	-65281

Figure 12. The graphic presenting the database with the tasks

categories

tasks

Live updates

Results are read-only

50

	id	name	description	category	type	firstDate	secondDate	daysRepeated
1	1	Task 1	Things to do	Category 1	Single task	2023-05-05	2023-05-05	1234567
2	2	Task 2		Category 3	Until task	2023-05-08	2023-05-12	1234588
3	3	Task 3		Category 2	Repeat task	2023-05-05	2023-07-31	1838587
4	4	Task 4	Extra information	Category 2	Until task	2023-05-09	2023-06-01	1234567

Composable functions and components

Composable functions are a feature of Jetpack Compose, a toolkit for building Android UI using Kotlin.

¹⁰ Create a Database instance. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab=https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#6>. Persist data with Room

In this app, many composable functions and components were used. They are presented below:

1. **Column** - arranges its children in a vertical column
2. **Row** - arranges its children in a horizontal row
3. **Image** - displays an image
4. **Icon** - displays an icon
5. **Text** - displays a text (as String)
6. **Scaffold** - provides a layout structure for an app screen
7. **Lazy Column** - a scrollable column
8. **Outlined Text Field** - a text input field
9. **Button** - provides a clickable button
10. **Floating Action Button** - provides a clickable circular button used for a primary action on a screen¹¹

They were crucial to creating a whole operation of the app. The use of a few of them is shown in the example below.

Figure 13. The graphic presenting the ListOfTasks() function

```
@SuppressLint("UnusedMaterialScaffoldPaddingParameter")
@Composable
fun ListOfTasks(
    navController: NavController,
    categoryViewModel: CategoryHomeViewModel = viewModel(factory = AppViewModelProvider.Factory),
    taskViewModel: TaskHomeViewModel = viewModel(factory = AppViewModelProvider.Factory),
    navigateEdit: (Int) -> Unit,
    navigateEditForTask: (Int) -> Unit,
    onDone: (Int) -> Unit,
) {
    val categoryHomeUiState by categoryViewModel.categoryHomeUiState.collectAsState()
    val taskHomeUiState by taskViewModel.taskHomeUiState.collectAsState()

    Scaffold(
        topBar = { SearchButton(onClick = {navController.navigate(Mains.Searching.name)}.)},
        bottomBar = { MainBar(navController = navController, firstDestination = stringResource("List of tasks")) },
        floatingActionButton = {
            FloatingActionButton({}) {
                HoldButton(onClick = {navController.navigate(Mains.CreatingTask.name)}, onLongClick = {navController.navigate(Mains.CreatingCategory.name)})
            }
        },
    ) {
        it.PaddingValues
        ListOfCategories(
            itemList = categoryHomeUiState.categoryList,
            navigateEdit = { navigateEdit(it.id) },
            itemList = taskHomeUiState.taskList,
            navigateEditForTask = { navigateEditForTask(it.id) },
            done = { onDone(it.id) }
        )
    }
}
```

In this case, a Scaffold and a FloatingActionButton are used. Nevertheless, there are other functions called, for instance, ListOfCategories() which displays information on the lack of categories as Text in a Row or a LazyColumn that includes the CategoryDesign() function, which displays categories in a pre-planned design.

¹¹ Android Studio. *Jetpack Compose Tutorial*. 05.05.2023.
<https://developer.android.com/jetpack/compose/tutorial> .

Figure 14. The graphic presenting the `ListOfCategories()` function¹²

```
@Composable
fun ListOfCategories(
    itemList: List<Category>,
    navigateEdit: (Category) -> Unit,
    itemList: List<Task>,
    navigateEditForTask: (Task) -> Unit,
    done: (Task) -> Unit
) {
    if (itemList.isEmpty() or ((itemList.size == 1) and (itemList.any {it.name == "Uncategorised"}) and (!itemList.any {it.category == "Uncategorised"}))) {
        Row(verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxWidth()
        ) {
            this: RowScope
            Text(
                text = stringResource(R.string.no_category),
                style = MaterialTheme.typography.body2,
                textAlign = TextAlign.Center
            )
        }
    }
    else {
        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
            this: LazyListScope
            items(items = itemList, key = {it.id}) { category ->
                if (category.name != "Uncategorised") CategoryDesign(
                    category = category,
                    navigateEdit = navigateEdit,
                    itemList = itemList,
                    navigateEditForTask = navigateEditForTask,
                    done = done
                )
            }
            else {
                if (itemList.any {it.category == "Uncategorised"}) {
                    CategoryDesign(
                        category = category,
                        navigateEdit = navigateEdit,
                        itemList = itemList,
                        navigateEditForTask = navigateEditForTask,
                        done = done
                    )
                }
            }
        }
    }
}
```

¹² Meghan Magical. 2022. Basic Android Kotlin Compose Training Woof
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-woof.git> (2023)

Figure 15. The graphic presenting the `CategoryDesign()` function¹³

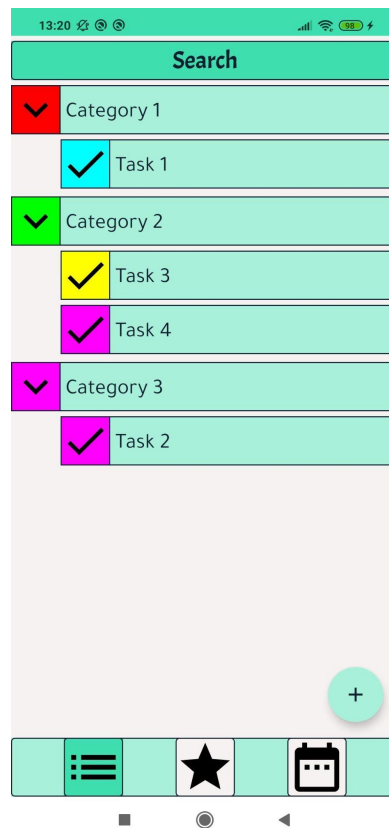
```
@Composable
fun CategoryDesign(
    category: Category,
    navigateEdit: (Category) -> Unit,
    itemList: List<Task>,
    navigateEditForTask: (Task) -> Unit,
    done: (Task) -> Unit
) {
    var expanded by remember { mutableStateOf(false) }

    Column ( verticalArrangement = Arrangement.spacedBy(5.dp)) {
        Row(
            Modifier
                .fillMaxWidth()
                .border(
                    width = 1.dp,
                    color = MaterialTheme.colors.onSurface
                )
                .background(color = MaterialTheme.colors.surface),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Image(imageVector = if (expanded) Icons.Filled.KeyboardArrowDown else Icons.Filled.KeyboardArrowRight,
                contentDescription = "arrow",
                modifier = Modifier
                    .size(50.dp)
                    .background(color = Color(category.colour))
                    .fillMaxHeight()
                    .clickable { expanded = !expanded }
                    .border(
                        width = 1.dp,
                        color = MaterialTheme.colors.onSurface
                    )
            )
            Row(Modifier
                .clickable { if (category.name != "Uncategorised") navigateEdit(category) }
                .fillMaxWidth()
            ) {
                Text(
                    text = category.name, //max = 27
                    color = MaterialTheme.colors.onPrimary,
                    style = MaterialTheme.typography.body1,
                    fontSize = 22.sp,
                    textAlign = TextAlign.Center,
                    modifier = Modifier
                        .padding(5.dp)
                )
            }
        }
    }
    if (expanded) ListOfTasksDisplay(itemList = itemList, currentCategory = category, navigateEdit = navigateEditForTask, done = done)
}
```

The result is presented below.

¹³ Meghan Magical. 2022. Basic Android Kotlin Compose Training Woof
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-woof.git> (2023)

Figure 16. The graphic presenting the display of the ListOfTasks menu on the phone



Dependency management

This technique was used to obtain different libraries with functions and values. They are downloaded and included in the programme at build time.

Figure 17. The graphic presenting the dependencies of Build Gradle^{14 15}

```
dependencies {
    implementation 'androidx.core:core-ktx:1.10.0'
    implementation 'androidx.compose.compiler:compiler:1.4.4'
    implementation 'androidx.lifecycle:lifecycle-runtime-ktx:2.6.1'
    implementation 'androidx.activity:activity-compose:1.7.0'
    implementation "androidx.compose.ui:ui:$compose_ui_version"
    implementation "androidx.compose.ui:ui-tooling-preview:$compose_ui_version"
    implementation 'androidx.compose.material:material:1.4.1'
    implementation 'androidx.navigation:navigation-runtime-ktx:2.5.3'
    implementation 'com.google.ar.sceneform:filament-android:1.17.1'
    implementation 'androidx.room:room-common:2.5.1'
    implementation 'androidx.room:room-ktx:2.5.1'
    testImplementation 'junit:junit:4.13.2'
    androidTestImplementation 'androidx.test.ext:junit:1.1.5'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'
    androidTestImplementation "androidx.compose.ui:ui-test-junit4:$compose_ui_version"
    debugImplementation "androidx.compose.ui:ui-tooling:$compose_ui_version"
    debugImplementation "androidx.compose.ui:ui-test-manifest:$compose_ui_version"
    implementation "androidx.navigation:navigation-fragment-ktx:2.5.3"
    implementation "androidx.navigation:navigation-ui-ktx:2.5.3"
    implementation "androidx.navigation:navigation-compose:2.5.3"
    implementation "androidx.room:room-runtime:$room_version"
    ksp "androidx.room:room-compiler:$room_version"
    implementation "androidx.room:room-ktx:$room_version"
    implementation 'com.kizitonwose.calendar:view:2.3.0'
    implementation 'com.kizitonwose.calendar:compose:2.3.0'
}
```

For instance, they allow access to the calendar library, which is used in the Calendar menu. They allow to creation of a database as well.

Figure 18. The graphic presenting the MyCalendar() function's importations

```
import com.kizitonwose.calendar.compose.HorizontalCalendar
import com.kizitonwose.calendar.compose.rememberCalendarState
import com.kizitonwose.calendar.core.*
```

¹⁴ Main components of Room. 05.05.2023.

<https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?continue=https%3A%2F%2Fdeveloper.android.com%2Fcourses%2Fpathways%2Fandroid-basics-compose-unit-6-pathway-2%23codelab-https%3A%2F%2Fdeveloper.android.com%2Fcodelabs%2Fbasic-android-kotlin-compose-persisting-data-room#3>. Persist data with Room

¹⁵ Kizito Nwose. *Calendar Compose Documentation*. 05.05.2023.

<https://github.com/kizitonwose/Calendar/blob/main/docs/Compose.md>

Figure 19. The graphic presenting the TaskDatabase() abstract class' importations

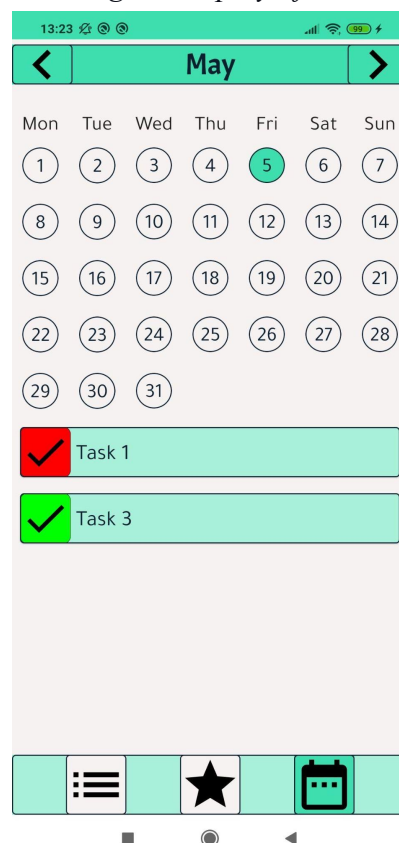
```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Task::class], version = 1, exportSchema = false)
abstract class TaskDatabase: RoomDatabase() {
```

Calendar and date operations

The calendar was used in the Calendar menu by the MyCalendar() function. It was obtained from the previously mentioned library. As requested, it is scrollable horizontally and allows you to pick a date to show tasks assigned to that day.

Figure 20. The graphic presenting the display of the Calendar menu on the phone



It includes CalendarState and is called as the built-in HorizontalCalendar() function with a few parameters responsible for the operation and the design.

Figure 21. The graphic presenting the initialization of the state value¹⁶

```
val state = rememberCalendarState(  
    startMonth = startMonth,  
    endMonth = endMonth,  
    firstVisibleMonth = currentMonth,  
    firstDayOfWeek = firstDayOfWeek,  
)
```

Figure 22. The graphic presenting the use of the built-in HorizontalCalendar() function¹⁷

```
HorizontalCalendar(  
    state = state,  
    dayContent = { day ->  
        Box(modifier.padding(10.dp)) { this: BoxScope  
            Day(  
                day, isSelected = selectedDate == day.date,  
                { day ->  
                    selectedDate = if (selectedDate == day.date) null else day.date  
                },  
                enabled = true  
            )  
        }  
    },  
    monthHeader = { this: ColumnScope  
        DaysOfWeekTitle(daysOfWeek = daysOfWeek())  
    }  
)
```

Date operations are also applied while presenting tasks in the Calendar, ToDoThisWeek, and ToDoToday menus. They allow you to filter specific tasks based on a specific date.

¹⁶ Kizito Nwose. *Calendar Compose Documentation*. 05.05.2023.
<https://github.com/kizitonwose/Calendar/blob/main/docs/Compose.md>

¹⁷ Ibid.

Figure 23. The graphic presenting the part of the `ToDoToday()` function

```
if (task.type == TypesTasks.singleTask) {
    if ((LocalDate.now().isAfter(LocalDate.parse(task.secondDate)))) {
        sortedList = sortedList.plus(
            taskTDTW(
                id = howManyTasks,
                task = task,
                date = LocalDate.now(),
                isOverdue = true
            )
        )
        howManyTasks++
    } else if (LocalDate.now() == LocalDate.parse(task.firstDate)) {
        sortedList = sortedList.plus(
            taskTDTW(
                id = howManyTasks,
                task = task,
                date = LocalDate.now()
            )
        )
        howManyTasks++
    }
} else if (task.type == TypesTasks.untilTask) {
    if (LocalDate.now().isAfter(LocalDate.parse(task.secondDate))) {
        sortedList = sortedList.plus(
            taskTDTW(
                id = howManyTasks,
                task = task,
                date = LocalDate.now(),
                isOverdue = true
            )
        )
        howManyTasks++
    } else if (LocalDate.now() == LocalDate.parse(task.secondDate).minusDays( daysToSubtract - 1)) {
        sortedList = sortedList.plus(
            taskTDTW(
                id = howManyTasks,
                task = task,
                date = LocalDate.now(),
                isOverdue = true,
                isDeadline = true
            )
        )
    }
}
```

Validation rules and conditional statements

Validation rules are crucial to operating properly. One of the rules is checking if `taskUiState` can be saved as `Task`. It must fulfill 3 requirements, as it has been planned. If `taskUiState` fulfills them, it can be saved. Otherwise, it might cause an error while saving.

Figure 24. The graphic presenting the `TaskUiState.isValid()` function¹⁸

```
fun TaskUiState.isValid() : Boolean {
    return name.isNotBlank() && type.isNotBlank() && (daysRepeated != 8888888)
}
```

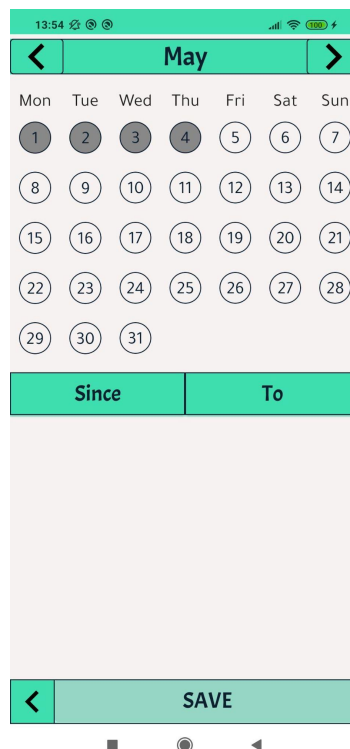
¹⁸ Dev Android. 2022. Basic Android Kotlin Compose Training Inventory App.
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git>
(2023)

Another validation rule is used while choosing two dates - “since” and “to”. A user cannot plan something in the past and the “since” date cannot be after the “to” date. Therefore other days are not clickable.

Figure 25. The graphic presenting the `HorizontalCalendar()` function from the `PickUpADate()` function¹⁹

```
HorizontalCalendar(
  state = state,
  dayContent = { day ->
    Box(Modifier.padding(10.dp)) { this: BoxScope
      Day(
        day, isSelected = selectedDate == day.date,
        { day ->
          selectedDate = if (selectedDate == day.date) null else day.date
        },
        enabled = (((day.date.isAfter(firstDate.minusDays( daysToSubtract: 1))) or (!toEnabled))
          and ((day.date.isBefore(secondDate.plusDays( daysToAdd: 1))) or (!sinceEnabled))
          and (day.date.isAfter(LocalDate.now().minusDays( daysToSubtract: 1))))
      )
    }
  },
),
```

Figure 26. The graphic presenting the display of the calendar while choosing a date with inactive days before the present day (05.05.2023)



¹⁹ Kizito Nwose. *Calendar Compose Documentation*. 05.05.2023.
<https://github.com/kizitonwose/Calendar/blob/main/docs/Compose.md>

Figure 27. The graphic presenting the display of the calendar while choosing a date with inactive days before the chosen “since” day (16.05.2023)

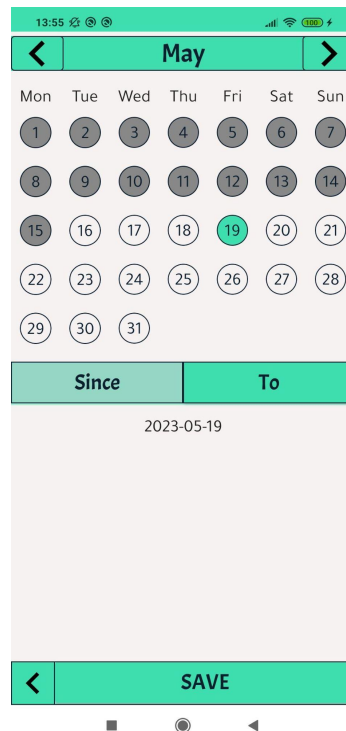
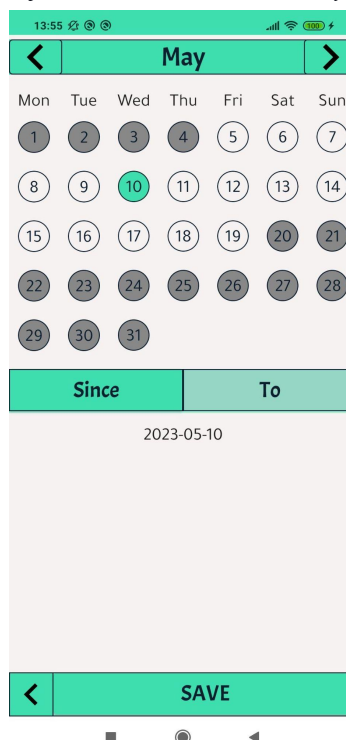


Figure 28. The graphic presenting the display of the calendar while choosing a date with inactive days after the chosen “To” day (19.05.2023)



Additionally, many conditional statements are used while presenting tasks in menus that use dates to filter tasks. (Figure 20.)

Navigation component

Navigation allows you to proceed to different menus that have been designed without any complications. There are 10 possible destinations. Each menu can proceed to a different one by using designed buttons which fulfills the navigation requirement.

Figure 29. The graphic presenting the part of the `NavigationHost()` function responsible for navigation²⁰

```
@Composable
fun NavigationHost(navController: NavHostController = rememberNavController()) {
    NavHost(navController = navController, startDestination = Mains.ToDoToday.name) {
        composable(route=Mains.ListOfTasks.name) { // it: NavBackStackEntry
            ListOfTasks(
                navController = navController,
                navigateEdit = { navController.navigate( route: "${CategoryEditDestination.route}/$it" ) },
                navigateEditForTask = { navController.navigate( route: "${TaskEditDestination.route}/$it" ) },
                onDone = {navController.navigate( route: "${TaskDeleteDestination.route}/$it" )}
            )
        }
        composable(route=Mains.ToDoToday.name) { // it: NavBackStackEntry
            ToDoToday(
                navController = navController,
                navigateDetails = { navController.navigate( route: "${TaskDetailsDestination.route}/$it" ) },
                onDone = {navController.navigate( route: "${TaskDeleteDestination.route}/$it" )}
            )
        }
        composable(route=Mains.ToDoThisWeek.name) { // it: NavBackStackEntry
            ToDoThisWeek(
                navController = navController,
                navigateDetails = { navController.navigate( route: "${TaskDetailsDestination.route}/$it" ) }
            )
        }
        composable(route=Mains.Calendar.name) { // it: NavBackStackEntry
            MyCalendar(navController = navController)
        }
        composable(route=Mains.CreatingCategory.name) { // it: NavBackStackEntry
            CreatingCategoryMenu(navController = navController)
        }
        composable(route=Mains.CreatingTask.name) { // it: NavBackStackEntry
            CreatingTaskMenu(navController = navController)
        }
        composable(route=Mains.Searching.name) { // it: NavBackStackEntry
            Searching(
                navController = navController,
                navigateDetails = { navController.navigate( route: "${TaskEditDestination.route}/$it" ) }
            )
        }
    }
}
```

Lists and arrays

Arrays were used to store permanent values, such as the names of weekdays. They allow access to these values by any function and therefore enable comparison easily. The implementation and the use are presented below.

²⁰ Dev Android. 2022. Basic Android Kotlin Compose Training Inventory App.
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git>
(2023)

Figure 30. The graphic presenting the arrays with the types of tasks and names of weekdays in two versions²¹

```
val typesTasksArray = arrayOf(
    TypesTasks.singleTask,
    TypesTasks.untilTask,
    TypesTasks.repeatTask
)

val repeatTaskAdditional = arrayOf(
    "Repeat every",
    "Until"
)

val weekDaysShortNames = arrayOf(
    "Mon",
    "Tue",
    "Wed",
    "Thu",
    "Fri",
    "Sat",
    "Sun"
)

val weekDaysNames = arrayOf(
    DayOfWeek.MONDAY,
    DayOfWeek.TUESDAY,
    DayOfWeek.WEDNESDAY,
    DayOfWeek.THURSDAY,
    DayOfWeek.FRIDAY,
    DayOfWeek.SATURDAY,
    DayOfWeek.SUNDAY
)
```

Figure 31. The graphic presenting the use of one of the arrays - weekDaysNames

```
var sortedList = listOf<taskTDTW>()
if (selectedDate != null) {
    val thisDay = selectedDate ?: LocalDate.now()
    val index = weekDaysNames.indexOf(thisDay.dayOfWeek) + 1
}
```

Lists were used to store values as well. Nevertheless, they are changed while the programme is running and the order of its elements does not matter (Figure 31.). In the Searching() function they were used to store chosen features.

Figure 32. The graphic presenting the lists in the Searching() function

```
var categories by remember { mutableStateOf(listOf<String>()) }
var types by remember { mutableStateOf(listOf<String>()) }
```

²¹ Android Studio. *Jetpack Compose Tutorial*. 05.05.2023.
<https://developer.android.com/jetpack/compose/tutorial> .

Figure 33. The graphic presenting the use of one of the lists - categories in the Searching() function

```
LazyColumn(verticalArrangement = Arrangement.spacedBy(3.dp)) { this: LazyListScope
    items(items = categoryHomeUiState.categoryList, key = {it.id}) { category ->
        if (category.name != "Uncategorised") CategoryShown(
            presentCategory = category,
            onClick = { categories = if (!categories.contains(category.name)) {
                categories.plus(category.name)
            } else {
                categories.minus(category.name)
            } },
            included = categories.contains(category.name)
        )
    }
}
```

WORD COUNT: 951

Sources

- 1) Android Studio. *Android Basics with Compose*. 05.05.2023.
<https://developer.android.com/courses/android-basics-compose/course>
- 2) Android Studio. *Jetpack Compose Tutorial*. 05.05.2023.
<https://developer.android.com/jetpack/compose/tutorial>
- 3) Dev Android. 2022. Basic Android Kotlin Compose Training Inventory App.
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git> (2023)
- 4) Kizito Nwose. 2019. Calendar. <https://github.com/kizitonwose/Calendar.git> (2023)
- 5) Kizito Nwose. *Calendar Compose Documentation*. 05.05.2023.
<https://github.com/kizitonwose/Calendar/blob/main/docs/Compose.md>
- 6) Meghan Magical. 2022. Basic Android Kotlin Compose Training Woof
<https://github.com/google-developer-training/basic-android-kotlin-compose-training-woof.git> (2023)